

# Wording for GB301, US296, US292, US291, and US283

Document #: P1983R0  
Date: 2019-11-15  
Project: Programming Language C++  
LWG  
Reply-to: Tim Song  
<[t.canens.cpp@gmail.com](mailto:t.canens.cpp@gmail.com)>

## § 1 Introduction

---

This paper provides combined wording for NB comments GB301, US296, US292, US291, and US283, all minor Ranges issues. It also resolves [LWG issue 3278](#).

## § 2 Wording

---

This wording is relative to [\[N4835\]](#).

### § 2.1 [GB301](#) Add `filter_view::pred()` accessor

Edit 24.7.4.2 [\[range.filter.view\]](#), class template `filter_view` synopsis, as indicated:

```
namespace std::ranges {
    template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
        requires view<V> && is_object_v<Pred>
    class filter_view : public view_interface<filter_view<V, Pred>> {
        [...]
    public:
        filter_view() = default;
        constexpr filter_view(V base, Pred pred);
        template<input_range R>
            requires viewable_range<R> && constructible_from<V, all_view<R>>
        constexpr filter_view(R&& r, Pred pred);

        constexpr V base() const;
+       constexpr const Pred& pred() const;

        [...]
    };

    template<class R, class Pred>
        filter_view(R&&, Pred) -> filter_view<all_view<R>, Pred>;
}
```

In 24.7.4.2 [\[range.filter.view\]](#), insert the following after p3:

```
constexpr const Pred& pred() const;
```

? *Effects:* Equivalent to: return \*pred\_;

## § 2.2 [US296](#) Converting constructor for `split_view::outer_iterator` is slightly wrong

[ Drafting note: LWG was not convinced that the constructor was wrong, but nonetheless agreed to change this for consistency with the rest of the Clause. ]

Edit 24.7.11.3 [\[range.split.outer\]](#), class template `split_view::outer_iterator` synopsis, as indicated:

```
namespace std::ranges {
    template<class V, class Pattern>
    template<bool Const>
    struct split_view<V, Pattern>::outer_iterator {

        [...]

        outer_iterator() = default;
        constexpr explicit outer_iterator(Parent& parent)
            requires (!forward_range<Base>);
        constexpr outer_iterator(Parent& parent, iterator_t<Base> current)
            requires forward_range<Base>;
        constexpr outer_iterator(outer_iterator<!Const> i)
            - requires Const && convertible_to<iterator_t<V>, iterator_t<const V>>;
            + requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

        constexpr value_type operator*() const;

        constexpr outer_iterator& operator++();
        constexpr decltype(auto) operator++(int) {
            if constexpr (forward_range<Base>) {
                auto tmp = *this;
                ++*this;
                return tmp;
            } else
                ++*this;
        }

        friend constexpr bool operator==(const outer_iterator& x, const
            outer_iterator& y)
            requires forward_range<Base>;

        friend constexpr bool operator==(const outer_iterator& x,
            default_sentinel_t);
    };
}
```

Edit 24.7.11.3 [\[range.split.outer\]](#) before p4 as indicated:

```
constexpr outer_iterator(outer_iterator<!Const> i)
- requires Const && convertible_to<iterator_t<V>, iterator_t<const V>>;
+ requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

- 4 *Effects:* Initializes `parent_` with `i.parent_` and `current_` with `std::move(i.current_)`.

## § 2.3 [US292](#) Incorrect constructor for `join_view::iterator`

Edit 24.7.10.3 [\[range.join.iterator\]](#), class template `join_view::iterator` synopsis, as indicated:

```
namespace std::ranges {
template<class V>
template<bool Const>
struct join_view<V>::iterator {

    [...]

    iterator() = default;
-   constexpr iterator(Parent& parent, iterator_t<V> outer);
+   constexpr iterator(Parent& parent, iterator_t<Base> outer);
    constexpr iterator(iterator<!Const> i)
        requires Const &&
            convertible_to<iterator_t<V>, iterator_t<Base>> &&
            convertible_to<iterator_t<InnerRng>,
                iterator_t<range_reference_t<Base>>>;

    [...]

};
}
```

Edit 24.7.10.3 [\[range.join.iterator\]](#) before p7 as indicated:

```
-   constexpr iterator(Parent& parent, iterator_t<V> outer)
+   constexpr iterator(Parent& parent, iterator_t<Base> outer);
```

- 7 *Effects:* Initializes `outer_` with `outer` and `parent_` with `addressof(parent)`; then calls `satisfy()`.

## § 2.4 [US291](#) `join_view::begin` requires mutable data

Edit 24.7.10.2 [\[range.join.view\]](#) as indicated:

```
namespace std::ranges {
template<input_range V>
requires view<V> && input_range<range_reference_t<V>> &&
    (is_reference_v<range_reference_t<V>> ||
     view<range_value_t<V>>)
class join_view : public view_interface<join_view<V>> {

    [...]

public:

    [...]

    constexpr auto begin() {
```

```

+ constexpr bool use_const = simple-view<V> &&
+ is_reference_v<range_reference_t<V>>;
- return iterator<simple-view<V>>{*this, ranges::begin(base_)};
+ return iterator<use_const>{*this, ranges::begin(base_)};
    }

    [...]
};

template<class R>
    explicit join_view(R&&) -> join_view<all_view<R>>;
}

```

## § 2.5 [US283](#) Specification of *has-arrow* concept is ill-formed

Edit 24.5.1 [\[range.utility.helpers\]](#) as indicated:

```

- template<input_iterator I>
+ template<class I>
    concept has-arrow = // exposition only
- is_pointer_v<I> || requires(I i) { i.operator->(); };
+ input_iterator<I> && (is_pointer_v<I> || requires(I i) { i.operator->
    (); });

```

## § 3 References

---

[N4835] Richard Smith. 2019. Working Draft, Standard for Programming Language C++. <https://wg21.link/n4835>